



Université Paris-Saclay

Faculté des Sciences

Projet d'Interface Interactive Avancée

Application de dessin

Présenté par : Wang Marc et Dragulinescu Alexandru

Groupe : LDD3-MAG-INFO

4 mai 2026

Table des matières

1	Introduction	2
2	Implémentation effective et code source	2
2.1	Architecture Modèle-Vue-Contrôleur	2
2.2	Gestion des formes et polymorphisme	2
2.3	Persistance des données (Sérialisation)	2
3	Tableau des fonctionnalités	2
4	Analyse ergonomique et Heuristiques	4
4.1	Heuristiques de Nielsen appliquées	4
4.2	Stratégies cognitives et principes WIMP	4
5	Conclusion	5

1 Introduction

Ce rapport présente l'implémentation finale de notre application de dessin interactive développée en C# WinForms. L'interface a été conçue selon des principes d'ergonomie modernes, privilégiant un design épuré pour réduire la charge cognitive de l'utilisateur. L'organisation MVC (Modèle-Vue-Contrôleur) adoptée permet une séparation claire entre la logique métier et le rendu graphique.

2 Implémentation effective et code source

2.1 Architecture Modèle-Vue-Contrôleur

L'application repose sur trois piliers fondamentaux :

- **Le Modèle (Model.cs)** : Gère la structure de données, notamment la liste des formes géométriques et les opérations de gestion (ajout, suppression, nettoyage).
- **La Vue (ZoneDessin.cs)** : Composant personnalisé gérant l'affichage graphique et capturant les événements utilisateur (souris et molette).
- **Le Contrôleur (Form1.cs)** : Orchestre l'interface utilisateur, les changements de modes d'action et les opérations d'entrée/sortie.

2.2 Gestion des formes et polymorphisme

Toutes les entités graphiques héritent de la classe abstraite **FormeGeo**. Chaque sous-classe (**Rectangle**, **Disque**, **Trait**) redéfinit les méthodes essentielles :

- **contient(Point p)** : Algorithme de détection de collision pour la sélection.
- **Cloner()** : Implémentation du patron de conception Prototype pour la duplication.

2.3 Persistance des données (Sérialisation)

La sauvegarde utilise le format JSON via la bibliothèque **System.Text.Json**. Pour pallier les limites de sérialisation des types natifs, nous avons mis en place des propriétés auxiliaires pour les couleurs (format ARGB) et utilisé des attributs **JsonDerivedType** pour conserver le type réel des objets lors du chargement.

3 Tableau des fonctionnalités

Ce tableau récapitule l'état d'avancement du projet par rapport au cahier des charges initial.

```
[
  {
    "$type": "disq",
    "Rayon": 40,
    "CouleurArgb": -1,
    "PinceauArgb": -16777216,
    "Position": { "IsEmpty": false, "X": 452, "Y": 326 }
  },
  {
    "$type": "rect",
    "Largeur": 100,
    "Hauteur": 50,
    "CouleurArgb": -1,
    "PinceauArgb": -16777216,
    "Position": { "IsEmpty": false, "X": 395, "Y": 217 }
  }
]
```

FIGURE 1 – Exemple d'un fichier Json de sauvegarde

Fonctionnalité	Description	État
Dessin de base	Tracé de lignes, rectangles et ellipses.	Réalisé
Déplacement	Sélection et déplacement libre des formes.	Réalisé
Choix des couleurs	Définition des couleurs de remplissage et trait.	Réalisé
Redimensionnement	Poignées de contrôle pour ajuster la taille.	Réalisé
Gestion des plans d'affichage	Placer un élément au premier plan ou au dernier plan.	Réalisé mais subtile : un double clique sur une figure lors de la sélection le place directement au premier plan
Sauvegarde éditable	Format JSON spécifique pour réédition ultérieure.	Réalisé
Outil de Zoom	Zoom avant/arrière fluide à la molette.	Réalisé mais ne zoom que dans le coin en haut à gauche
Duplication	Possibilité de cloner une forme sélectionnée.	Fonctionnalité supplémentaire et réalisé

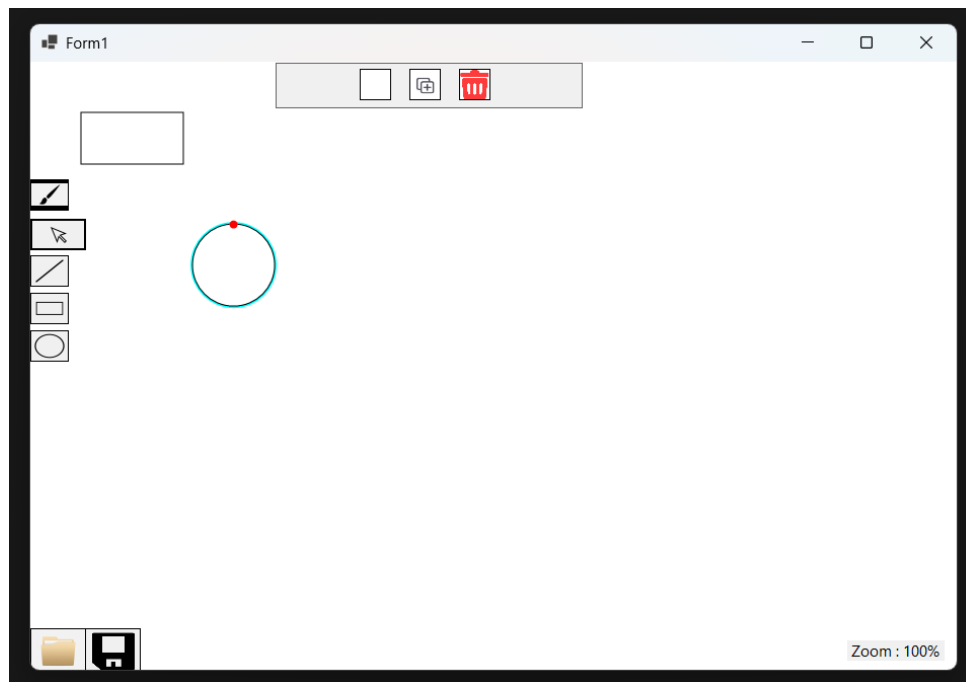


FIGURE 2 – Interface finale

4 Analyse ergonomique et Heuristiques

L'ergonomie logicielle a pour but de maximiser les performances au travail en adaptant la machine à l'humain. Pour ce projet, nous avons appliqué plusieurs principes issus du cours.

4.1 Heuristiques de Nielsen appliquées

L'utilisation d'heuristiques de conception permet de réduire la charge cognitive et de produire une interface standardisée sans recourir à des panels d'utilisateurs coûteux.

- **Visibilité de l'état du système** : Le système informe continuellement l'utilisateur. L'affichage dynamique du niveau de zoom (ex : "Zoom : 100%") en bas de l'écran offre un feedback immédiat sur l'état de la vue.
- **Concordance avec le monde réel** : Nous utilisons des métaphores visuelles standardisées. Les icônes de disquette pour la sauvegarde et de dossier pour l'ouverture facilitent l'incorporation dans le réseau de connaissances préalable de l'utilisateur.
- **Reconnaissance plutôt que rappel** : Toutes les commandes de dessin sont constamment affichées dans la barre d'outils latérale. Cela réduit l'effort mnésique car l'utilisateur n'a pas à mémoriser de combinaisons de touches complexes.
- **Esthétique et design minimaliste** : Conformément au principe d'éliminer l'information non pertinente, l'interface est épurée pour ne laisser que les outils nécessaires à la tâche.

4.2 Stratégies cognitives et principes WIMP

L'application respecte les standards des interfaces WIMP (Windows, Icons, Menus, Pointers).

- **Principe WYSIWYG** : "What You See Is What You Get". L'affichage à l'écran correspond exactement aux formes géométriques qui seront produites et sauvegardées.
- **Accentuation visuelle** : Pour attirer l'attention sans surcharger l'utilisateur, nous utilisons l'encadrement (couleur cyan) et des poignées de couleur (rouge) pour marquer la forme sélectionnée et les points de contrôle.

5 Conclusion

L'application remplit l'intégralité des objectifs fixés. L'ajout des fonctionnalités de zoom et de redimensionnement dynamique offre une précision accrue, répondant ainsi aux besoins spécifiques identifiés dans nos scénarios d'usage. La structure modulaire du code facilite l'ajout futur de nouvelles formes ou fonctionnalités.